Wrapper Klassen

13.11.2025

Plan für die Woche

- x
- y
- z
- a
- b

Plan für die Woche

Montag	Dienstag	Mittwoch	Donnerstag	Freitag
• StringBuilder	• DateTimeUtils	• DateTimeUtils	• Wrapper-Klassen	• Selbsteinschätzung • Quiz • Var-args

Lernziel

- Verstehen, was Wrapper-Klassen sind
- Unterschied zwischen Wrapper-Klassen und ihren primitiven Datentypen verstehen

Wrapper-Klassen

Grundlagen

- sind Klassen, die primitive Datentypen in Objekte verpacken („wrappen“)
- dadurch können primitive Datentypen auch für ArrayList genutzt werden

Wrapper-Klassen

Primitiver Datentype	Wrapper-Klasse
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Boolean-Konstruktor

```
Boolean booleanE2 = new Boolean(true);              //true
Boolean booleanE = new Boolean("TRUE");             //true
Boolean booleanE_1 = new Boolean("tRUe");           //true
Boolean booleanE_2 = new Boolean("false");          //false
Boolean booleanE_3 = new Boolean("FALSE");          //false
Boolean booleanE_4 = new Boolean("false2348");      //false
Boolean booleanE_5 = new Boolean("2348");           //false
Boolean booleanE_4 = new Boolean("true28");         //false
```

Byte - Konstruktor

Muss davor geschrieben werden! Sonst Compilerfehler

```
Byte byteE = new Byte((byte)1);
Byte byteE2 = new Byte("127");
Byte byteE3 = new Byte("129");      //Exception: Wertebereich zu groß
```

Die 1 muss explizit gecastet werden, da Zahlen in Java standardmäßig immer ein int sind.

Short - Konstruktor

Muss davor geschrieben werden! Sonst Compilerfehler

```
Short shortE = new Short((short)1);
Short shortE2 = new Short("10");
Short shortE3 = new Short("13058230583405");   //Exception
```

Die 1 muss explizit gecastet werden, da Zahlen in Java standardmäßig immer ein int sind.

Integer und Long - Konstruktor

```
Integer integer = new Integer(10);
Integer integer2 = new Integer("10");

Long longE = new Long(10);
Long longE2 = new Long("10");
```

Warum muss beim Long nicht mehr explizit gecastet werden?

Float - Konstruktor

Doubles sind auch valide als Parameter!

```
Float floatE = new Float(10.0);
Float floatE2 = new Float(10.0F);
Float floatE3 = new Float("10");
Float floatE3_2 = new Float("10.0");
Float floatE3_3 = new Float("10.0F");
Float floatE3_4 = new Float("10.0D");
```

Warum geht das mit dem double?
Weil der Konstruktor doubles ohne casten erlaubt

Double - Konstruktor

Floats sind auch valide als Parameter

```
Double doubleE = new Double(10.0);
Double doubleE2 = new Double(10.0F);
Double doubleE3 = new Double("10.0F");
Double doubleE3_2 = new Double("10.0D");
Double doubleE3_3 = new Double("10");
Double doubleE3_4 = new Double("10.0");
```

Character - Konstruktor

```
Character c = new Character('a');     //muss ein char drinnen stehen
```

Methoden

Mittels Integer.intValue() kriegt man den primitiven Wert der Wrapper-Klasse

- **pareseInt(String s):** Wandelt eine String-Repräsentation eines primitiven Datentyps (z. B. int) in den entsprechenden Wert um.

```
int num = Integer.parseInt("123");
```

- **valueOf(String s):** Wandelt eine String-Repräsentation in das entsprechende Wrapper-Objekt um.

```
Integer num = Integer.valueOf("123");
```

- **toString():** Gibt den Wert des Wrapper-Objekts als String zurück.

```
Integer num = 10;
System.out.println(num.toString());
```

- **compareTo():** Vergleicht zwei Wrapper-Objekte.

```
Integer num1 = 10;
Integer num2 = 20;
System.out.println(num1.compareTo(num2));
```

Aufgabe

Schreibe ein Java-Programm, das einen Benutzer nach einer Zahl fragt, diese mit Integer.parseInt() umwandelt und diese dann mit sich selbst multipliziert. Was passiert, wenn man einen Buchstaben eingibt? Wie kann man das umgehen?

Autoboxing und Unboxing

- Autoboxing bedeutet, dass Java automatisch primitive Datentypen in ihre Wrapper-Klassen umwandelt

- Unboxing ist das automatische auspacken des primitiven Werts aus einem Wrapper-Objekt

```
Integer a = 5;  // Autoboxing
int b = a + 10; // Unboxing und Berechnung
System.out.println(b); // 15
```

Constant Pooling

- für ganzzahlige Wrapper-Datentypen (Byte, Short, Integer, Long) existiert ein sogenannter Pool
- der Bereich reicht von -128 bis 127
- Was heißt das?

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // true, da aus dem Pool genommen
```

Constant Pooling

- für ganzzahlige Wrapper-Datentypen (Byte, Short, Integer, Long) existiert ein sogenannter Pool
- der Bereich reicht von -128 bis 127
- Was heißt das?

```
Integer a = 200;
Integer b = 200;
System.out.println(a == b); // false, da außerhalb des Poolbereichs
```

Wie funktioniert es technisch?

- bei einer Zuweisung über das **Autoboxing** prüft die JVM: Liegt der Wert im Poolbereich?
  - Ja: Objekt aus dem Pool wird genommen
  - Nein: Neues Objekt wird erzeugt
- Schreibt man Integer i = new Integer(15); so wird immer ein neues Objekt erzeugt

Wie funktioniert es technisch?

- bei einer Zuweisung über das **Autoboxing** prüft die JVM: Liegt der Wert im Poolbereich?
  - Ja: Objekt aus dem Pool wird genommen
  - Nein: Neues Objekt wird erzeugt
- Schreibt man Integer i = new Integer(15); so wird immer ein neues Objekt erzeugt

Wie funktioniert es technisch?

- bei einer Zuweisung über das **Autoboxing** prüft die JVM: Liegt der Wert im Poolbereich?
  - Ja: Objekt aus dem Pool wird genommen
  - Nein: Neues Objekt wird erzeugt
- Schreibt man Integer i = new Integer(15); so wird immer ein neues Objekt erzeugt

Warum braucht man das?

- Speicher sparen: Häufig verwendete Zahlen werden nicht mehrfach im Speicher gehalten
- Effizienz: Vergleiche mit == sind im Poolbereich möglich, außerhalb sonst mittels .equals()

```
Integer x = 100;
Integer y = 100;
Integer a = 128;
Integer b = 128;
System.out.println(x == y); // true
System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
```